

Submission Worksheet

Submission Data

Course: IT114-007-F2025

Assignment: IT114 - Milestone 3 - RPS

Student: Ryan C. (rc728)

Status: Submitted | **Worksheet Progress:** 79%

Potential Grade: 6.00/10.00 (60.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 12/12/2025 4:56:45 AM

Updated: 12/12/2025 4:56:45 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-3-rps/grading/rc728>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-milestone-3-rps/view/rc728>

Instructions

1. Refer to Milestone3 of [Rock Paper Scissors](#)
 1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the Milestone3 branch
 1. `git checkout Milestone3`
 2. `git pull origin Milestone3`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. ``git commit -m "adding PDF"`
 3. `git push origin Milestone3`
 4. On Github merge the pull request from Milestone3 to main
7. Upload the same PDF to Canvas
8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`

Section #1: (1 pt.) Core Ui

Progress: 100%

≡ Task #1 (0.50 pts.) - Connection/Details Panels

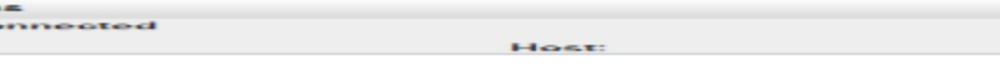
Progress: 100%

📁 Part 1:

Progress: 100%

Details:

- Show the connection panel with valid data

- 
- M185-Client
- Rooms
- Not Connected
- Host:
- 127.0.0.1
- Port:
- 3001
- Next

[illegible]

 Saved: 12/12/2025 4:28:52 AM

Progress: 100%

Your Response:

User details: After the client connects, the server would create a `ServerThread` and sends back a `CLIENT_ID` payload with the Client's information such as id and name using `sendClientId()`. The client then receives this in the `processPayload` method and passes it to the `processClientData` method which sets the user's id and name and put it into `knownClients`. Then the client calls the UI callback `onReceiveClientID()` and then implements updates to the details panel and score so the username and id are displayed

Task #2 (0.50 pts.) - Ready Panel

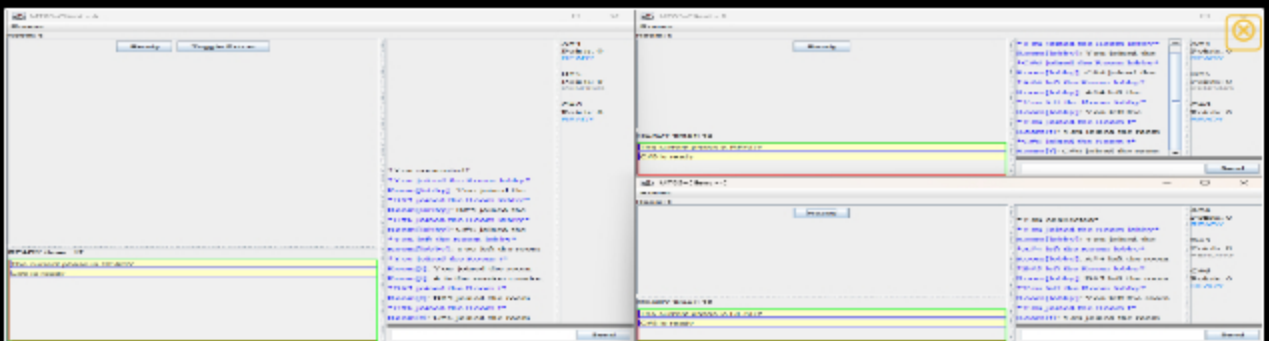
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)



this shows the ready button being used by two clients and one stays pending and unready

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI
- Briefly explain the code flow from receiving READY data and updating the UI

Your Response:

Code flow for marking READY from the UI: when the user clicks on the ready button in ReadyView, the button's action listener calls the sendReady method in client where in sendReady, the client create a ReadyPayload and sets the payload type to READY which is then sent to the server with sendToServer. in the server, the method processPayload handles the READY case and calls handleReady() which is implemented in the BaseGameRoom. This method allows for the player to be marked as ready and starts the ready timer and broadcast the ready status to all the clients using sendReadyStatus.

Code flow for receiving READY data and updating the UI: When the server calls sendReadyStatus, each client receives a ReadyPayload. in the processPayload method in client, the READY or SYNC_READY case will forward the payload to processReadyStatus. ProcessReadyStatus will then update the client's ready status and broadcast the status to all the clients.

then update the matching user object in the knownClients map and calls all UI callbacks that implement IReadyEvent through the onReceiveReady method. the view that implements IReadyEvent uses the info to update the ready indicator next to each player.



Saved: 12/12/2025 4:45:35 AM

Section #2: (2 pts.) Project Ui

Progress: 100%

≡ Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

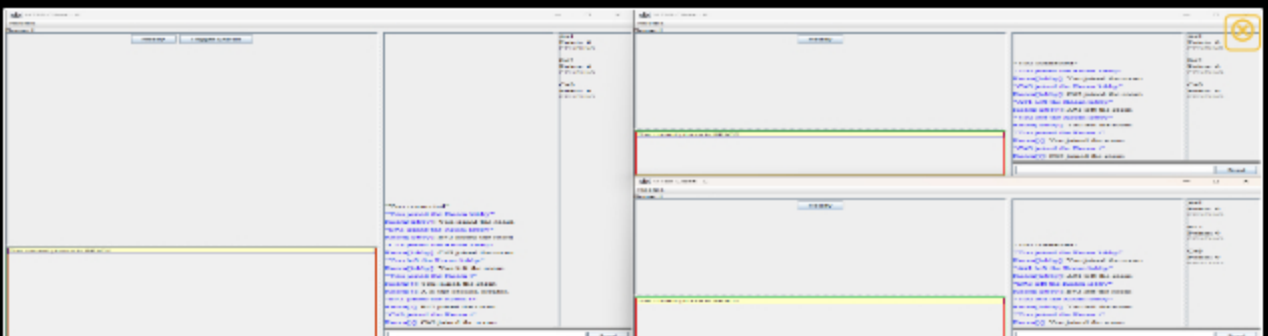
- Show the username and id of each user
- Show the current points of each user
- Users should appear in score order, sub-sort by name when ties occur
- Pending-to-pick users should be marked accordingly
- Eliminated users should be marked accordingly

📁 Part 1:

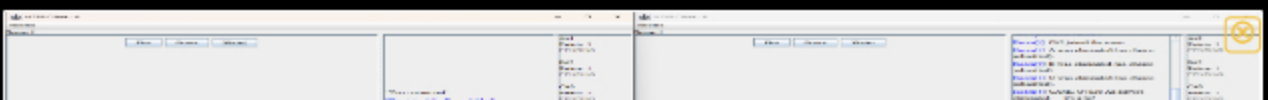
Progress: 100%

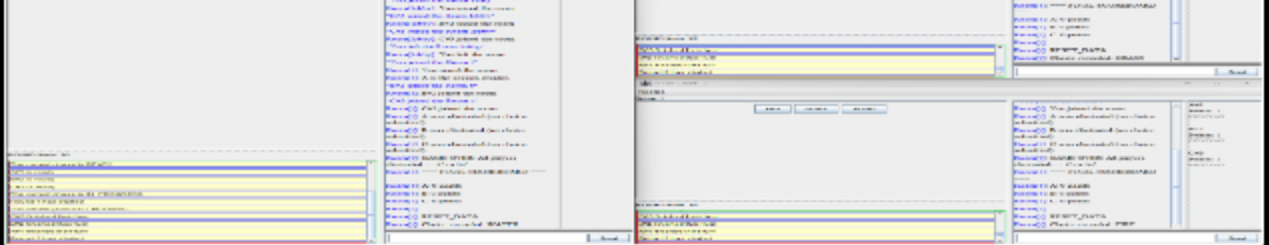
Details:

- Show various examples of points (3+ clients visible)
 - Include code snippets showing the code flow for this from server-side to UI
- Show that the sorting is maintained across clients
 - Include code snippets showing the code that handles this
- Show various examples of the pending-to-pick indicators
 - Include code snippets showing the code flow for this from server-side to UI
- Show various examples of elimination indicators
 - Include code snippets showing the code flow for this from server-side to UI

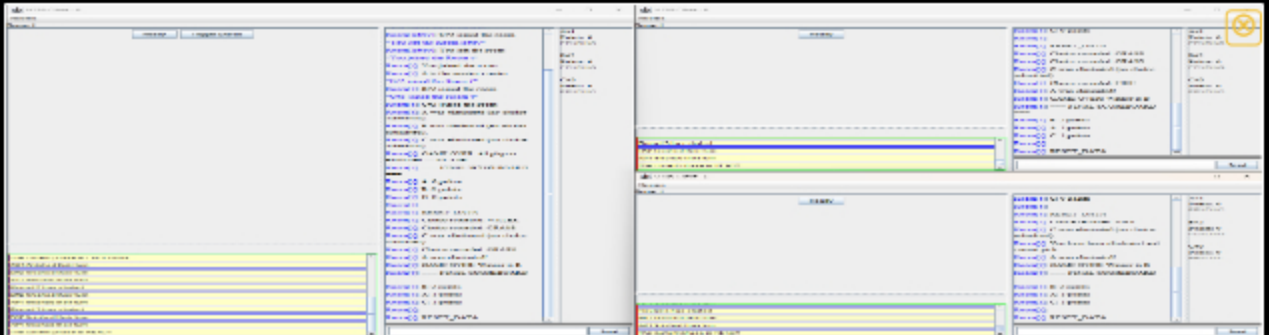


this shows the beginning before ready with all the clients in the room with 0 points

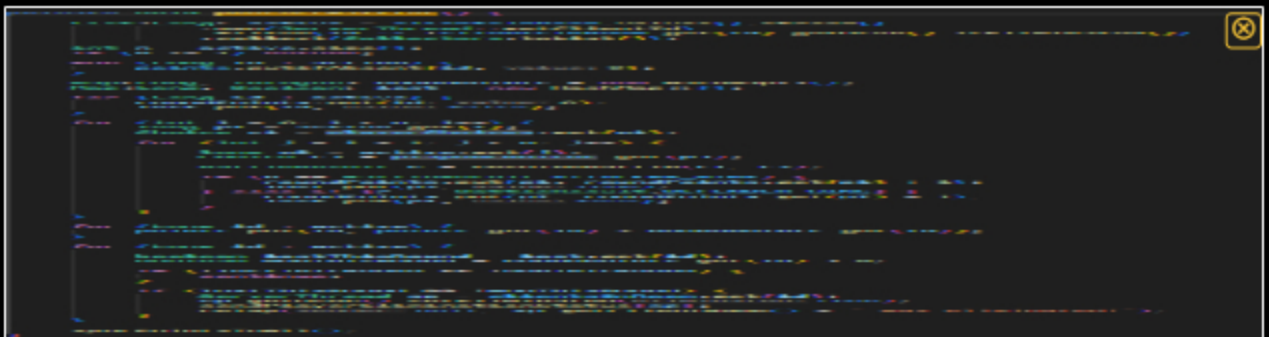




This shows the mid game where all of them got a point because they all eliminated one person



This shows the end game where the points are reset but you can see the final scoreboard with one person having 2 who won



This shows the processBattles() where it calculates and broadcasts points

```
//Rc728 12/11/25
public boolean sendPlayerPoints(long clientId, int points) {
    PointsPayload rp = new PointsPayload();
    rp.setPoints(points);
    rp.setClientId(clientId);
    return sendToClient(rp);
}
```

Severthread would wrap the points in a payload

```
//Rc728 12/11/25
private void processPoints(Payload payload) {
    if (!(payload instanceof PointsPayload)) {
        error(message: "Invalid payload subclass for processCardAdd");
        return;
    }
    PointsPayload pp = (PointsPayload) payload;
    long targetId = pp.getClientId();
    int points = pp.getPoints();

    if (targetId == Constants.DEFAULT_CLIENT_ID) {
        // Reset all
        knownClients.values().forEach(cp -> cp.setPoints(1));
        passToUICallback(type: IPointsEvent.class, e -> e.onPointsUpdate(Constants.DEFAULT_CLIENT_ID, -1));
    } else if (knownClients.containsKey(targetId)) {
        knownClients.get(targetId).setPoints(points);
        passToUICallback(type: IPointsEvent.class, e -> e.onPointsUpdate(targetId, points));
    }
}
```

The Client receives the pointsPayload and updates the user models

[illegible]

It marks the players having taken their turn

```
//R6-728 12/11/25
//RoundStart
protected void onRoundStart() {
    LoggerUtil.INSTANCE.info(message: "onRoundStart() start");
    ClientInRoom.values().forEach(p -> p.setTookTurn(tookTurn: false));
    sendResetTurnStatus();
    resetRoundTimer();
    round++;

    sendGameEvent("Round = " + round + " has started");

    // Reset choices only for surviving players
    for (ServerThread p : ClientInRoom.values()) {
        long id = p.getClientId();

        if ((p.isPlayer() && !p.isEliminated()) || !
            playerChoices.put(id, value: null);
    }

    clientPong(Pong.CHECKING);
    startRoundTimer();

    LoggerUtil.INSTANCE.info(message: "onRoundStart() end");
}
```

make it so everyone will be pending again the next round which resets their turn

```
//RC728 12/11/25
public boolean sendResetTurnStatus() {
    ReadyPayload rp = new ReadyPayload();
    rp.setPayloadType(PayloadType.RESET_TURN);
    return sendToClient(rp);
}

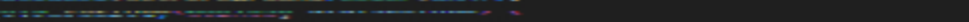
public boolean sendTurnStatus(long clientId, boolean didTakeTurn) {
    return sendTurnStatus(clientId, didTakeTurn, quiet: false);
}

public boolean sendTurnStatus(long clientId, boolean didTakeTurn, boolean quiet) {
    ReadyPayload rp = new ReadyPayload();
    rp.setPayloadType(quiet ? PayloadType.SYNC_TURN : PayloadType.TURN);
    rp.setClientId(clientId);
    rp.setReady(didTakeTurn);
    return sendToClient(rp);
}
```

the serverThread sends the TURN and RESET_TURN payloads

[illegible]

The Client updates `didTakeTurn` and notifies the UI



UI shows all the indicators such as pending, scores, user and ids, and ready or picked

[illegible]

The server Marks eliminated and broadcasts the eliminationpayload

```
//Rc728 12/11/25
public boolean sendPlayerEliminationStatus(long clientId, boolean isEliminated) {
    EliminationPayload p = new EliminationPayload(clientId, isEliminated);
    return send(p);
}
```

The Serverthread wraps the eliminationstatus of the player in a eliminationpayload

```

01 public void ProcessEliminationStatus(Payload payload) {
02     if (payload instanceof EliminationPayload) {
03         ErrorMessage errorMessage = new ErrorMessage("processEliminationStatus");
04         return;
05     }
06     EliminationPayload ep = (EliminationPayload) payload;
07     long targetId = ep.getClientId();
08     boolean isEliminated = ep.isEliminated();
09     LoggerUtil.info(LINK_LOGGER, "processEliminationStatus: clientId="
10         + targetId + ", isEliminated=" + isEliminated + ", knownClientIds="
11         + StringUtils.join(KnownClientIds.getInstance().getKnownClientIds(), ", "));
12     Set<String> knownClients = StringUtils.join(KnownClientIds.getInstance().getKnownClientIds(), ", ");
13     if (isEliminated) {
14         LoggerUtil.info(LINK_LOGGER, "processEliminationStatus: clientId="
15             + targetId + ", isEliminated=" + isEliminated + ", knownClientIds="
16             + StringUtils.join(KnownClientIds.getInstance().getKnownClientIds(), ", ")
17             + "isEliminated=" + "isEliminated" + ", notEliminated=");
18     } else {
19         LoggerUtil.info(LINK_LOGGER, "processEliminationStatus: clientId="
20             + targetId + ", isEliminated=" + isEliminated + ", knownClientIds="
21             + StringUtils.join(KnownClientIds.getInstance().getKnownClientIds(), ", ")
22             + "isEliminated=" + "isEliminated" + ", notEliminated=");
23     }
24     LoggerUtil.info(LINK_LOGGER, "processEliminationStatus: clientId="
25         + targetId + ", isEliminated=" + isEliminated + ", knownClientIds="
26         + StringUtils.join(KnownClientIds.getInstance().getKnownClientIds(), ", ")
27         + "isEliminated=" + "isEliminated" + ", notEliminated=");
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }

```

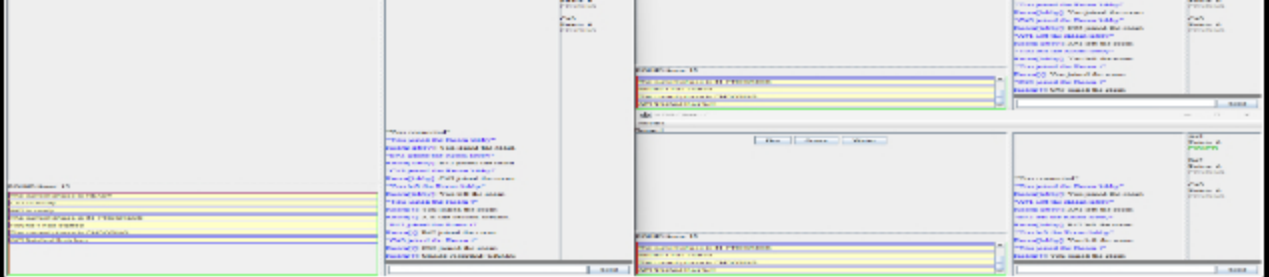
The client updates the model and it notifies the UI

```
//Rc728 12/11/25
public void setEliminated(boolean isEliminated) {
    eliminated = isEliminated;

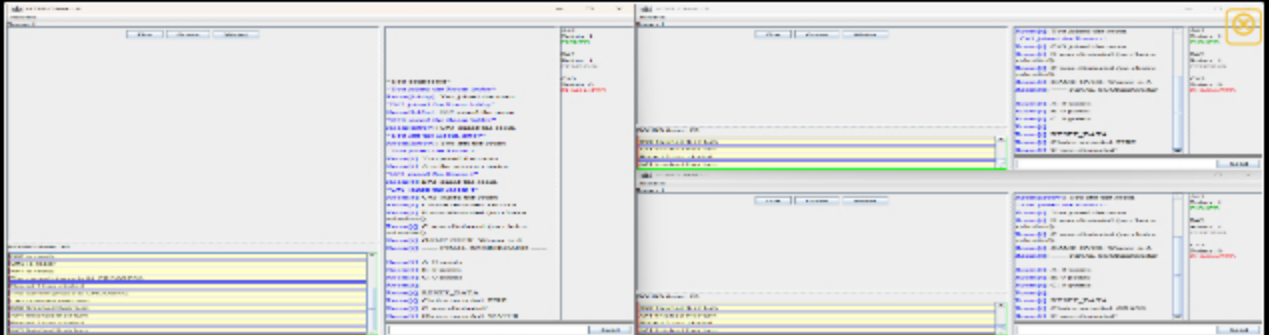
    if (isEliminated) {
        statusLabel.setText(t: "ELIMINATED");
        statusLabel.setForeground(Color.RED);
    } else {
        setPending();
    }
}
```

The UI marks the eliminated user





This shows pending with one of the clients having picked their option



This shows the eliminated status after client C got eliminated in the first round



Saved: 12/11/2025 8:45:22 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting
- Briefly explain the code flow for server-side to UI of pending-to-pick indicators
- Briefly explain the code flow for server-side to UI of elimination indicators

Your Response:

Code flow for points updates from server-side to the UI: after every round, the server will calculate new scores and send a `PointsPayload` for each player by using the `sendPlayerPoints`. The Client then receives this payload in the `processPoints` method which updates the player points in its `knownClients` list. Then the Client calls the UI callback `onPointsUpdate` which will refresh the scoreboard display.

Code flow for user list sorting: The UI would read all the user objects from the `Client.INSTANCE.getKnownClients()` which would sort them by points with the highest amount of points being first, then it would go by name if there is a tie in score. Then it renders the list in that order so every client will display the same ranking.

Code flow for server-side to UI of pending-to-pick: At the start of every round, the server will reset everyone's turn state and send `RESET_TURN`. In the client `processResetTurn()` it will set `didTakeTurn` to false and calls `onTookTurn()` which the UI will use to mark those users to Pending until another choice is received.

Code flow for server-side to UI of elimination indicators: When a player is eliminated, the server marks them as eliminated and sends an EliminationPayload. The client then receives this payload and processEliminationStatus() updates the User.setEliminated and the client sends the UI onUserEliminated which the UI uses to visually mark the eliminated user as eliminated

 Saved: 12/11/2025 8:45:22 PM

≡ Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

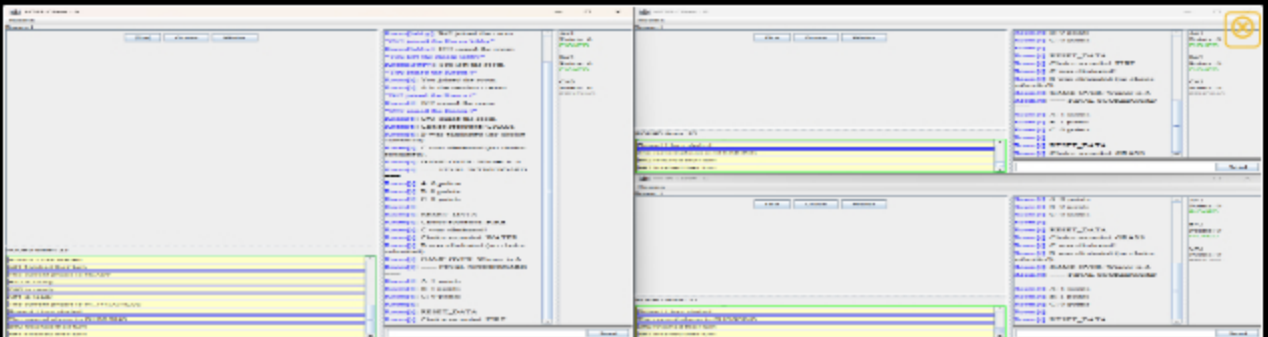
- Show the status of users picking choices
- Show the battle resolution messages from Milestone 2
 - Include messages about elimination
- Show the countdown timer for the round

Part 1:

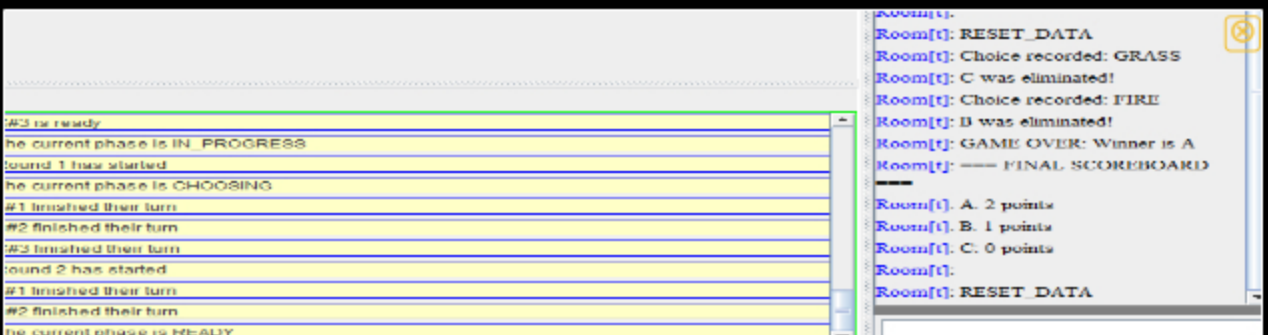
Progress: 100%

Details:

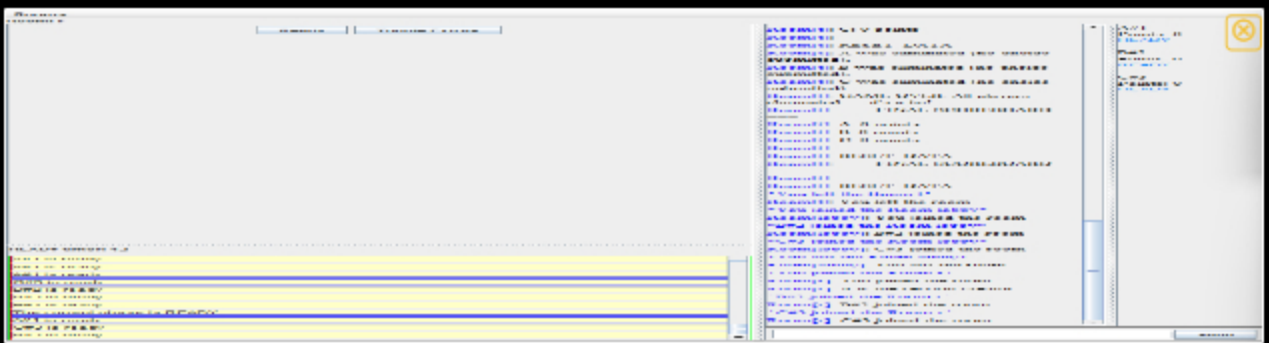
- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI



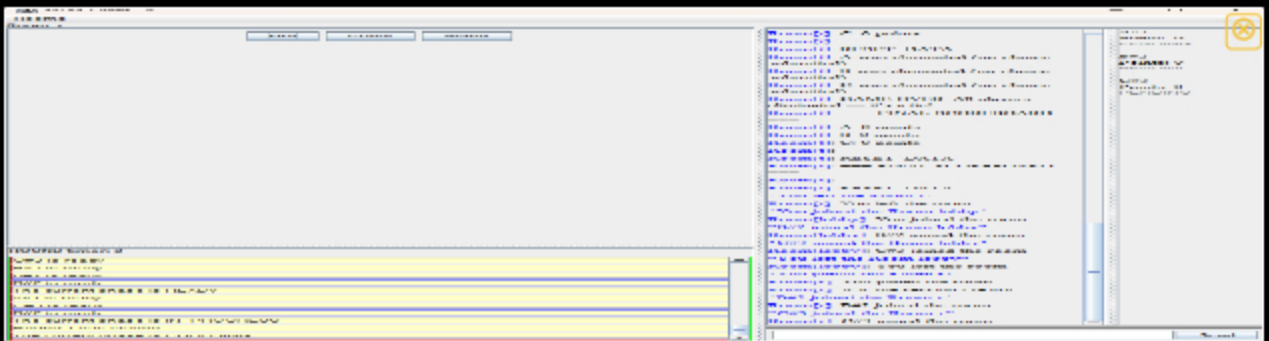
shows two clients with the PICKED status and with different choices made and one with pending with no choices made



on the left shows the game event panel showing players finishing their turns the right shows the battle resolution with score



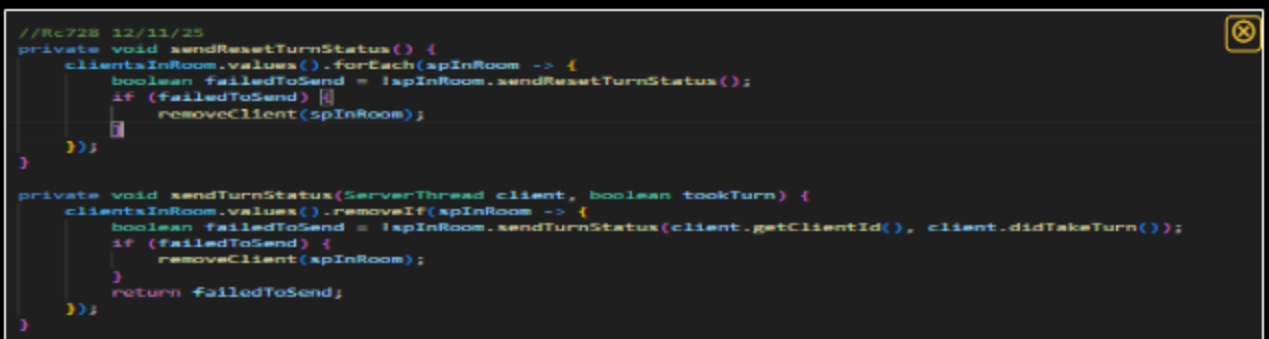
This shows the ready countdown during the ready phase



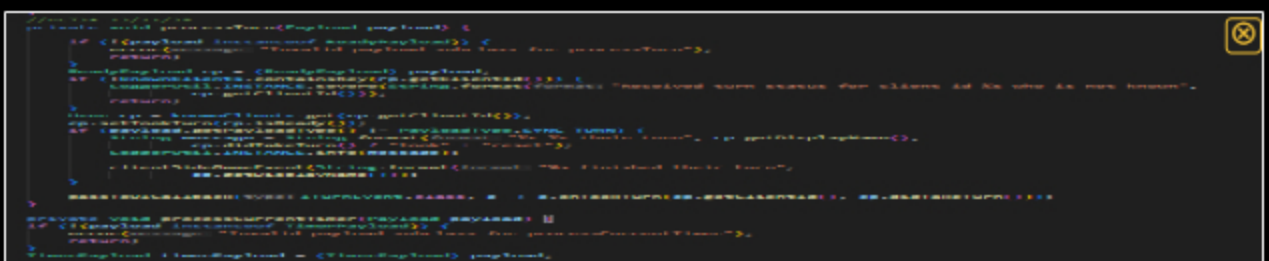
This shows the round countdown during the choosing phase



the player picks a choice



this broadcasts finish turn or reset pending



```

@Override
protected void onRoundEnd() {
    // Reset round timer
    resetRoundTimer();
    // Update user state and the UI
    for (ServerThread p : clientsInRoom.values()) {
        long id = p.getClientId();
        boolean isEliminated = p.getUser().isEliminated();
        boolean madeChoice = playerChoices.get(id) != null;
        if (!isEliminated && !madeChoice) {
            ServerThread eliminatedPlayer = clientsInRoom.get(id);
            if (eliminatedPlayer != null && !eliminatedPlayer.getUser().isEliminated()) {
                eliminatedPlayer.getUser().setEliminated(true);
                sendPlayerEliminationStatus(eliminatedPlayer);
                relay(sender: null, p.getClientName() + " was eliminated (no choice submitted).");
            }
        }
    }
}

```

this updates the user state and the UI

```

@Override
protected void onRoundEnd() {
    // Reset round timer
    resetRoundTimer();
    // Update user state and the UI
    for (ServerThread p : clientsInRoom.values()) {
        long id = p.getClientId();
        boolean isEliminated = p.getUser().isEliminated();
        boolean madeChoice = playerChoices.get(id) != null;
        if (!isEliminated && !madeChoice) {
            ServerThread eliminatedPlayer = clientsInRoom.get(id);
            if (eliminatedPlayer != null && !eliminatedPlayer.getUser().isEliminated()) {
                eliminatedPlayer.getUser().setEliminated(true);
                sendPlayerEliminationStatus(eliminatedPlayer);
                relay(sender: null, p.getClientName() + " was eliminated (no choice submitted).");
            }
        }
    }
}

```

this resolves the battles and calculate and awards points

```

//rc728 12/11/25
@Override
protected void onRoundEnd() {
    LoggerUtil.INSTANCE.info(message: "onRoundEnd() start");
    resetRoundTimer();

    for (ServerThread p : clientsInRoom.values()) {
        long id = p.getClientId();
        boolean isEliminated = p.getUser().isEliminated();
        boolean madeChoice = playerChoices.get(id) != null;

        if (!isEliminated && !madeChoice) {
            ServerThread eliminatedPlayer = clientsInRoom.get(id);
            if (eliminatedPlayer != null && !eliminatedPlayer.getUser().isEliminated()) {
                eliminatedPlayer.getUser().setEliminated(true);
                sendPlayerEliminationStatus(eliminatedPlayer);
                relay(sender: null, p.getClientName() + " was eliminated (no choice submitted).");
            }
        }
    }
}

```

this part of onRoundEnd allows for no choice submitted eliminations

```

//rc728 12/11/25
@Override
protected void onRoundEnd() {
    LoggerUtil.INSTANCE.info(message: "onRoundEnd() start");
    resetRoundTimer();

    for (ServerThread p : clientsInRoom.values()) {
        long id = p.getClientId();
        boolean isEliminated = p.getUser().isEliminated();
        boolean madeChoice = playerChoices.get(id) != null;

        if (!isEliminated && !madeChoice) {
            ServerThread eliminatedPlayer = clientsInRoom.get(id);
            if (eliminatedPlayer != null && !eliminatedPlayer.getUser().isEliminated()) {
                eliminatedPlayer.getUser().setEliminated(true);
                sendPlayerEliminationStatus(eliminatedPlayer);
                relay(sender: null, p.getClientName() + " was eliminated (no choice submitted).");
            }
        }
    }
}

```

this shows the final scoreboard and the game over messages

```

// timer handlers
//rc728 12/11/25
private void startRoundTimer() {
    roundTimer = new TimerEvent(durationInSeconds: 30, () -> onRoundEnd());
    roundTimer.setTickCallback((time) -> {
        System.out.println("Round Time: " + time);
        sendCurrentTime(TimerType.ROUND, time);
    });
}

private void resetRoundTimer() {
    if (roundTimer != null) {
        roundTimer.cancel();
        roundTimer = null;
        sendCurrentTime(TimerType.ROUND, -1);
    }
}

```

this handles the phase timer where it counts down from 30 seconds

```

//rc728 12/11/25
protected void resetReadyTimer() {
    // Reset ready timer
    resetReadyTimer();
}

```



```

        + (readyTimer != null) {
            readyTimer.cancel();
            readyTimer = null;
            sendCurrentTime(TimerType.READY, -1);
        }
    }
}

```

broadcast the time to all the players

```

//rc728 12/11/25
public boolean sendCurrentTime(TimerType timerType, int time) {
    TimerPayload tp = new TimerPayload();
    tp.setTime(time);
    tp.setTimerType(timerType);
    return sendToClient(tp);
}

```

wraps the timer info in a timerpayload

```

//rc728 12/11/25
private void processCurrentTimer(Payload payload) {
    if (!(payload instanceof TimerPayload)) {
        error(message: "Invalid payload subclass for processCurrentTimer");
        return;
    }
    TimerPayload timerPayload = (TimerPayload) payload;

    passToUICallback(type: ITimeEvents.class, e ->
        e.onTimerUpdate(timerPayload.getTimerType(), timerPayload.getTime()));
}

```

The client receives the timerpayload

```

//rc728 12/11/25
@Override
public void onTimerUpdate(TimerType timerType, int time) {
    if (time >= 0) {
        timerText.setText(String.format(format: "%s timer: %s", timerType.name(), time));
    } else {
        timerText.setText(text: " ");
    }
    timerText.setVisible(aFlag: true);
}

```

this updates the timer label to actually show the timer going down in the UI

 Saved: 12/11/2025 10:04:47 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for generating these messages and getting them onto the UI

Your Response:

picking a choice: when a player clicks the choice button it sends a Choicepayload to the server where the gameroom handles the choice where it sets their tookTurn to true and calls the sendTurnStatus which sends a Readypayload of type turn to every Client. The client then processes the turn and updates the users tookTurn flag and sends it to the UI where the UI changes the user from PENDING to turn picked.

battle resolution and elimination messages: at the end of a round, the servers processbattles method will compare each player's choices and will decide who won or lost and will calculate and update their points and marks the loser as eliminated. When someone is eliminated, the server sends an EliminationPayload along with a chat message that they player was eliminated. In the client class, processEliminationStatus method updates the player elimination status for the UI which would show he status that they were eliminated.

round countdown timer when a round starts, the gameroom startRoundTimer creates a timedEvent where it counts down and every tick that happens calls the sendCurrentTime() which will then wrap the value in a timerpayload and send it to all the clients. Th is is where each each client then will receive a payload and call the onTimerUpdate which will update hte UI so all the clients can see the same countdown.



Saved: 12/11/2025 10:04:47 PM

≡ Task #3 (0.67 pts.) - Game Area

Progress: 100%

Details:

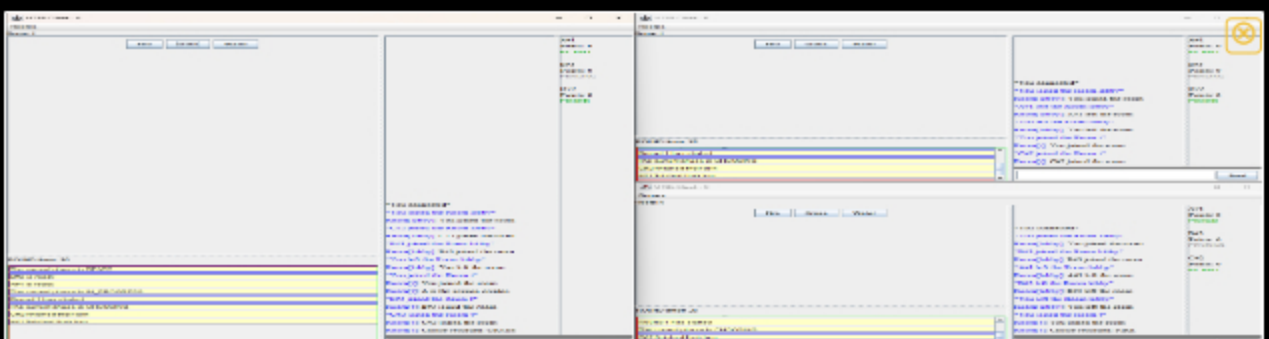
- UI should have components to allow the user to select their choice

🖼️ Part 1:

Progress: 100%

Details:

- Show various examples of selections across clients (3+ clients visible)
- Show the code related to sending choices upon selection
- Show the code related to showing visually what was selected



```
//rc728 12/11/25
private JButton makeButton(String label, String msg) {
    JButton b = new JButton(label);
    b.addActionListener(_ -> {
        try {
            Client.INSTANCE.sendChoice(msg);
        } catch (IOException e) {
            e.printStackTrace();
        }
    });
    return b;
}
```

```
//rc/28 12/11/25
public void sendChoice(String input) throws IOException {
    if (input == null) return;

    ChoicePayload.Choice choice = PICK_MAP.get(input.toLowerCase());
    if (choice == null) {
        LoggerUtil.INSTANCE.warning("Invalid choice: " + input);
        return;
    }

    sendChoice(choice);
}

private void sendChoice(ChoicePayload.Choice choice) throws IOException {
    ChoicePayload cp = new ChoicePayload(choice);
    sendToServer(cp);
}
```

```
//rc728 12/11/25
case CHOICE:
    try {
        ((GameRoom) currentRoom).handleChoiceAction(
            this, (ChoicePayload) incoming);
    } catch (Exception e) {
        sendMessage(Constants.DEFAULT_CLIENT_ID,
            message: "You must be in a GameRoom to choose an action");
    }
    break;
```

[illegible]

```
//rc728 12/11/25
public void setTurn(boolean didTakeTurn) {
    if (eliminated) return;

    if (didTakeTurn) {
        statusLabel.setText(t: "PICKED");
        statusLabel.setForeground(Color.GREEN.darker());
    }
}
```

```

    } else {
        setPending();
    }
}

```

This marks the visual for a choice being picked by changing pending to picked

 Saved: 12/11/2025 10:37:28 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for selecting a choice and having it reach the server-side
- Briefly explain the code flow for receiving the selection for the current player to update the UI

Your Response:

code flow for selecting a choice and having it reach the server-side: when a button is press like fire, water, or grass the button calls `client.INSTANCE.sendChoice()` where the client will turn the word into a `choicePayload` and send it to the server where in the server goes through it cases to `CHOICE` payload and call the `handleChoiceAction()` in `gameroom` where it stores the players choice for the round and marks the player as taken their turn.

For receiving the selection and updating the UI after a choice is saved, the `gameroom's handleChoiceAction` method sends a turn update to everyone by calling the `sendTurnStatus` method. The client then receives this message in `client processTurn` which updates the user's `didtaketurn` flag. The UI then uses this to refresh the display so that the player now shows as having picked an option instead of pending.

 Saved: 12/11/2025 10:37:28 PM

Section #3: (4 pts.) Project Extra Features

Progress: 50%

≡ Task #1 (2 pts.) - Extra Choices

Progress: 100%

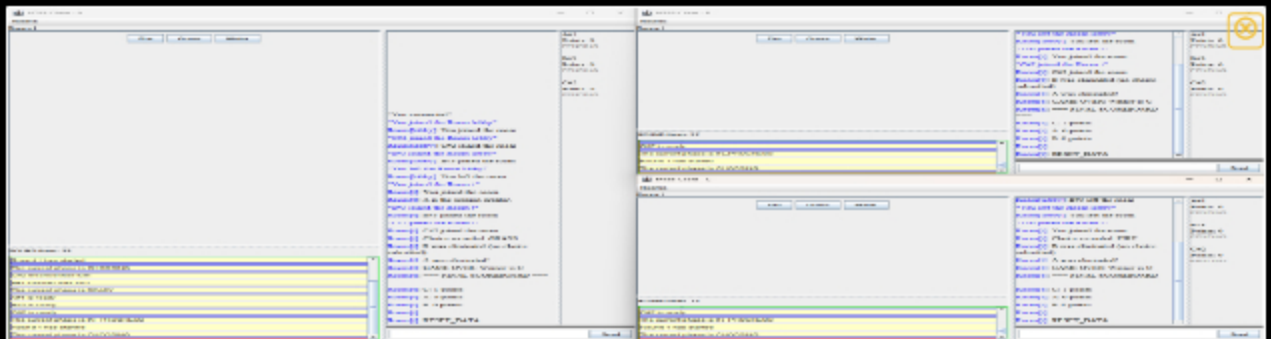
Details:

- Setting should be toggleable during Ready Check by session creator
 - (Option 1) Extra choices are available during the full session
 - (Option 2) Only activate extra options at different stages (i.e., last 3 players remaining)
- There should be at least 2 extra options for rps-5

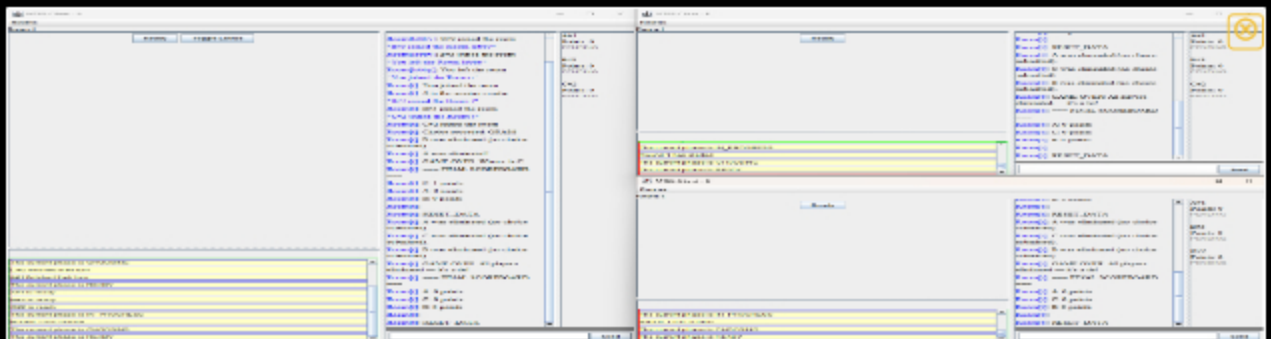
≡ Part 1:

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
 - Show the related code that makes this interactable only for the host
- Show the play screen with the extra options available
 - Show the related code for the UI and handling of these extra options (including battle logic)



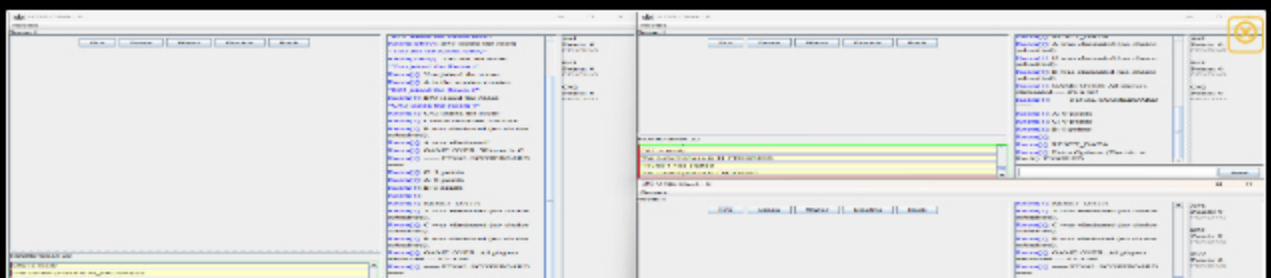
this shows a regular game with only 3 options



This show how only the host has the option to toggle extra

Room[t]: RESET_DATA
Room[t]: Extra Options (Electric + Rock): ENABLED

This shows that the extra options has been enabled in chat





This shows a game where the extra options have been enabled

```
// 12/12/24
private BattleResult resolveBattle(Choice a, Choice b) {
    if (a == b) return BattleResult.TIE;
    switch (a) {
        case FIRE:
            if (b == Choice.GRASS) return BattleResult.ATTACKER_WINS;
            if (b == Choice.ELECTRIC) return BattleResult.TIE; // NEW
            return BattleResult.DEFENDER_WINS;

        case WATER:
            if (b == Choice.FIRE || b == Choice.ROCK) return BattleResult.ATTACKER_WINS;
            return BattleResult.DEFENDER_WINS;

        case GRASS:
            if (b == Choice.WATER || b == Choice.ROCK) return BattleResult.ATTACKER_WINS;
            if (b == Choice.ELECTRIC) return BattleResult.TIE; // NEW
            return BattleResult.DEFENDER_WINS;

        case ELECTRIC:
            if (b == Choice.WATER) return BattleResult.ATTACKER_WINS;
            if (b == Choice.FIRE || b == Choice.GRASS) return BattleResult.TIE; // NEW
            return BattleResult.DEFENDER_WINS;

        case ROCK:
            if (b == Choice.FIRE || b == Choice.ELECTRIC) return BattleResult.ATTACKER_WINS;
            return BattleResult.DEFENDER_WINS;
    }
    return BattleResult.TIE;
}
```

This shows the battle logic the also incorporate the extra moves

```
public void toggleExtraOptions(ServerThread sender) {
    if (sender.getClientId() != creatorId) {
        sender.sendMessage(Constants.DEFAULT_CLIENT_ID,
            "Only the session creator can toggle extra options.");
        return;
    }
    if (currentPhase != Phase.READY) {
        sender.sendMessage(Constants.DEFAULT_CLIENT_ID,
            "You may only toggle during READY phase.");
        return;
    }
    extrasEnabled = !extrasEnabled;
    relay(sender, null, "Extra Options (Electric + Rock): " +
        (extrasEnabled ? "ENABLED" : "DISABLED"));
    broadcastChoiceMode();
}
```

this shows the playview where the extra choices are also stored as extra buttons

```
public void toggleExtraOptions(ServerThread sender) {
    if (sender.getClientId() != creatorId) {
        sender.sendMessage(Constants.DEFAULT_CLIENT_ID,
            "Only the session creator can toggle extra options.");
        return;
    }
    if (currentPhase != Phase.READY) {
        sender.sendMessage(Constants.DEFAULT_CLIENT_ID,
            "You may only toggle during READY phase.");
        return;
    }
    extrasEnabled = !extrasEnabled;
    relay(sender, null, "Extra Options (Electric + Rock): " +
        (extrasEnabled ? "ENABLED" : "DISABLED"));
    broadcastChoiceMode();
}
```

allows for the host to turn on and off the toggle feature for extra options

```
initial class ReadyView extends JPanel implements Serializable {
    private JButton toggleExtraOptions;

    public ReadyView() {
        setBackground(Color.WHITE);
        addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                if (e.getSource() == toggleExtraOptions) {
                    toggleExtraOptions.setText("Toggle Extras");
                }
            }
        });
        toggleExtraOptions = new JButton("Toggle Extras");
        toggleExtraOptions.addActionListener(this);
        toggleExtraOptions.setEnabled(false);
        Client.getInstance().sendMessage(Constants.DEFAULT_CLIENT_ID,
            "Toggle Extra Options: " + (extrasEnabled ? "ENABLED" : "DISABLED"));
    }

    public void enableToggleExtraOptions() {
        toggleExtraOptions.setEnabled(true);
    }

    public void disableToggleExtraOptions() {
        toggleExtraOptions.setEnabled(false);
    }
}
```

this creates the ready and toggle button in the lobby which only the host can see

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling these options including how it's handled during the battle logic
- Note which option you went with in terms of activating the choices

Your Response:

I went with option 1 where the host toggles the extra options in the ready phase

Host option to toggle extras: the server marks the first player who joins the room as creatorId and send a HOST_INFO payload to all the clients. With this creatorId, only the client that matches the id gets treated like the host where in ReadyView, the onHostIdentified method makes it so that the toggle extra button is only visible for the host and the other players in the room can't see it or press it. When the host clicks on the button, the client sends the special chat command of "/toggleExtras" where the serverThread will see it and call on the toggleExtraOptions method in gameroom.

Code related to handling these options and how battle logic uses them: in the gameroom for toggleExtraOptions method, the server will first check if the caller is the creator of the room and check what the current phase is and if it is READY. If both these checks pass, it will then flip the extrasEnabled boolean where it prints a chat message that extra options have been enabled or disabled and then calls the broadcastChoiceMode method which sends a CHOICE_MODE payload to every client. In the client, processPayload handles the CHOICE_MODE and calls onExtraModeChanged() on any IExtraModeEvent listeners. Playview then uses this to either show or hide the Electric and Rock button so all the players in the room have the same move set. In the battle logic which is the resolveBattle method in Gameroom, the extrasEnabled flag is checked where if the extras are disabled and if a player tries to use either the Electric or Rock option, the match up will force a tie. When extras is enabled however, the full rps-5 rules are ran where it gives the real wins and losses for the Electric and Rock options.



Saved: 12/12/2025 4:05:21 AM

≡ Task #2 (2 pts.) - Choice cooldown

Progress: 0%

Details:

- Setting should be toggleable during Ready Check by session creator
- The choice on cooldown must be disable on the UI for the User

Part 1:

Progress: 0%

Details:

- Show the Ready Check screen with the option for the host (3+ clients must be visible)
 - Show the related code that makes this interactable only for the host
- Show a few examples of the play screen with the choice on cooldown
 - Show the related code for the UI and handling of the cooldown and server-side enforcing it



Missing Caption



Saved: 12/4/2025 1:24:39 PM

⇒ **Part 2:**

Progress: 0%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling and enforcing the cooldown period (include how this is recorded per user and reset when applicable)

Your Response:

Missing Response



Saved: 12/4/2025 1:24:39 PM

Section #4: (2 pts.) Project General Requirements

Progress: 0%

≡ Task #1 (1 pt.) - Away Status

Progress: 0%

Details:

- Clients can mark themselves away and be skipped in turn flow but still part of the game
- The status should be visible to all participants
- A message should be relayed to the Game Events Panel (i.e., Bob is away or Bob is no

longer away)

- The user list should have a visual representation (i.e., grayed out or similar)

🖼️ Part 1:

Progress: 0%

Details:

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic



Missing Caption



Saved: 12/4/2025 1:24:39 PM

☰ Part 2:

Progress: 0%

Details:

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores the user from turn/round logic

Your Response:

Missing Response



Saved: 12/4/2025 1:24:39 PM

☰ Task #2 (1 pt.) - Spectators

Progress: 0%

Details:

- Spectators are users who didn't mark themselves ready

- Optionally you can include a toggle on the Ready Check page
- The can see all chat but are ignored from turn/round actions and can't send messages
- Spectators will have a visual representation in the user list to distinguish them from other players
- A message should be relayed to the Game Events Panel that a spectator joined (i.e., during an in-progress session)

Part 1:

Progress: 0%

Details:

- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending messages (server-side)
- Show the spectator's view of the session
- Show the code related to the spectator seeing the session data (including things participants won't see)



Missing Caption



Saved: 12/4/2025 1:24:39 PM

Part 2:

Progress: 0%

Details:

- Briefly explain the code flow for the spectator logic from server-side and to UI
- Briefly explain how the server-side ignores the user from turn/round logic
- Briefly explain the logic that prevents spectators from sending a message
- Briefly explain the logic that shares extra details to the spectator (information normal participants won't see)

Your Response:

Missing Response



Saved: 12/4/2025 1:24:39 PM

Section #5: (1 pt.) Misc

Progress: 100%

≡ Task #1 (0.33 pts.) - Github Details

Progress: 100%

📁 Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history

Milestone3 #10

524RC wants to merge 3 commits into main from RC728

Conversation | Commits | Checks | Files changed

Commits on Dec 11, 2025

- everything so far (524RC committed 3 hours ago)

Commits on Dec 12, 2025

- proposal (524RC committed 4 hours ago)
- Milestone3 done (524RC committed 1 minute ago)

This shows the commits made for milestone3



Saved: 12/12/2025 4:50:04 AM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the Pull Request for Milestone3 to main (should end in `/pull/#`)

URL #1

<https://github.com/524RC/RC728-IT114-pull-2025/>



URL

<https://github.com/524RC/RC728>



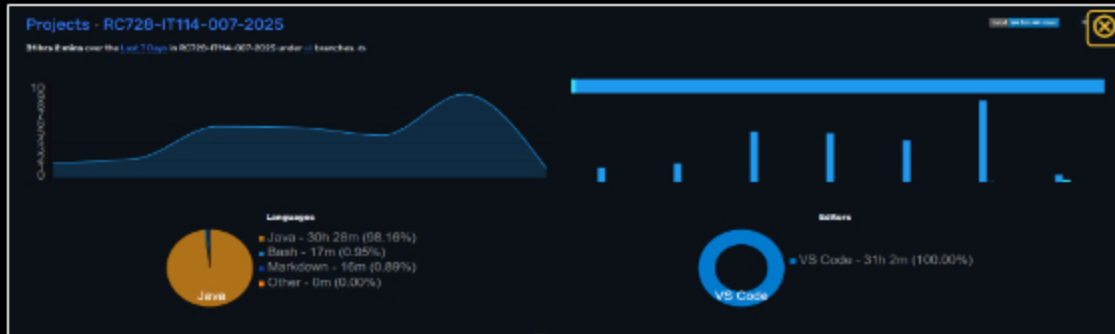
Saved: 12/12/2025 4:50:04 AM

📁 Task #2 (0.33 pts.) - WakaTime - Activity

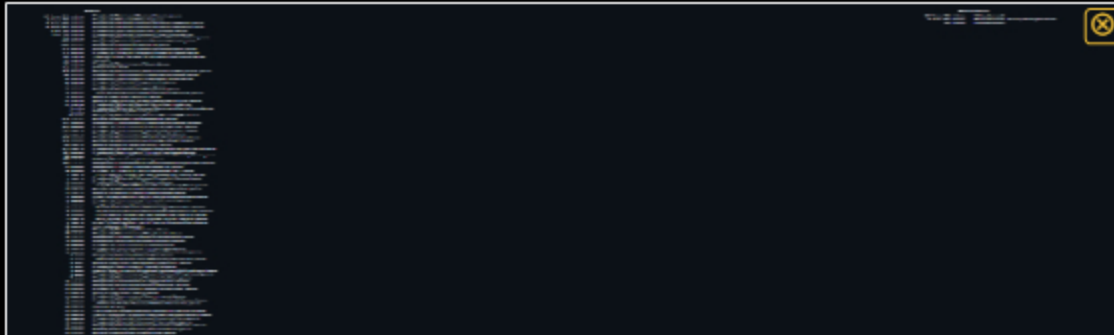
Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



Top part of waka time



Bottom Part of waka time

Saved: 12/12/2025 4:52:28 AM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned more about UI interfaces and code flow. From this project I learned a lot about the flow of the code like how you have to go from the gameroom to the serverthread to the client to the UI which was really interesting and good to learn.



Saved: 12/12/2025 4:53:48 AM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment was implementing the buttons and the extra buttons to be able to choose rock paper scissors or for my one which is fire water grass electric rock. It was the easiest to figure out and implement into my code.



Saved: 12/12/2025 4:55:14 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

FillerThe hardest part of the assignment, which also leads into what I learned, was getting the elimination to work and appear in the UI for players to be marked eliminated. I really struggled with this one trying to figure out why my elimination payload werent reaching my UI. Here I learned more about the flow of code and was able to trace the pathing of where the payloads where going.



Saved: 12/12/2025 4:56:45 AM